

Estruturas de Dados e Análise de Algoritmos

Mestrado Profissional em Computação Aplicada

Prof. Dr. Eng. Eduardo Takeo Ueda

Exercício-Programa 1

Implementação da Árvore Rubro-Negra

Data limite de entrega: 16 de dezembro de 2025

Uma árvore rubro-negra (ou vermelho-preta, do inglês *red-black tree*) é uma árvore binária de busca balanceada que, de forma simplificada, realiza operações de inserção e remoção de maneira inteligente, garantindo que a estrutura permaneça aproximadamente balanceada. Assim, a altura da árvore é mantida em $O(\log n)$, onde n representa o número de nós da árvore.

Essa estrutura foi proposta em 1972 por Rudolf Bayer, professor emérito da Universidade Técnica de Munique, sob o nome de “árvores binárias B simétricas” (*symmetric binary B-trees*). Posteriormente, em 1978, ela foi renomeada para “árvores rubro-negras” em um artigo de Leonidas John Guibas e Robert Sedgwick.

Uma árvore rubro-negra satisfaz as seguintes propriedades, conhecidas como propriedades rubro-negras:

- (1) Cada nó da árvore é **vermelho** ou **preto**.
- (2) A raiz da árvore é sempre **preta**.
- (3) Toda folha (NULL) da árvore é **preta**.
- (4) Se um nó da árvore é **vermelho**, então ambos os seus filhos são **pretos**.
- (5) Para cada nó da árvore, todos os caminhos deste nó até suas folhas contêm o mesmo número de nós **pretos**.

Na Figura 1, temos um exemplo de árvore rubro-negra, ou seja, uma árvore binária de busca que satisfaz as propriedades descritas anteriormente.

Uma árvore rubro-negra pode ser facilmente desbalanceada em decorrência das operações de inserção e remoção. Para corrigir tal desbalanceamento, é necessário aplicar um esquema de rotações, para restabelecer o balanceamento da estrutura.

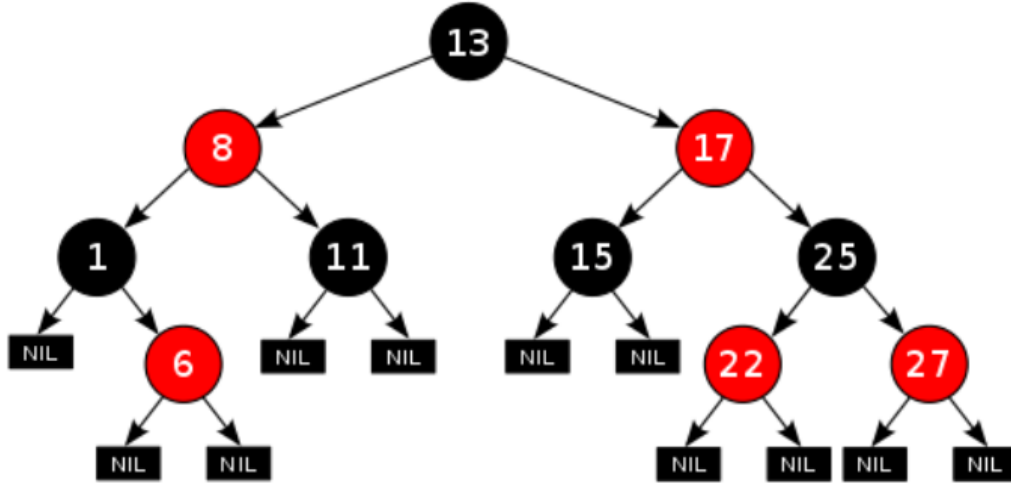


Figura 1: Árvore rubro-negra.

Existem quatro tipos de rotação: rotação simples à direita, rotação simples à esquerda, rotação dupla à direita e rotação dupla à esquerda.

Para manter o balanceamento de uma árvore rubro-negra, a cada **inserção** é executada uma série de verificações com o intuito de assegurar que suas propriedades permaneçam válidas. Caso alguma propriedade seja violada, torna-se necessário realizar **rotações** e **alterações de cores** dos nós, a fim de restabelecer o balanceamento da estrutura.

Sempre que um novo elemento é inserido na árvore, ele recebe inicialmente a cor **vermelha**. Se o nó inserido for a raiz, sua cor é imediatamente alterada para **preta**, de modo a satisfazer a propriedade (2). Caso o nó inserido não seja a raiz, deve-se verificar a propriedade (4), observando a cor do seu “pai” e do seu “tio”, para determinar se será necessário aplicar **rotações** e/ou **alterações de cores** adicionais.

A **remoção** em uma árvore rubro-negra inicia-se com uma busca e remoção semelhantes às realizadas em uma árvore binária de busca convencional. Em seguida, caso alguma propriedade rubro-negra seja violada, a árvore deve ser rebalanceada por meio de **rotações** e **alterações de cores** apropriadas.

De forma resumida, o processo pode ser descrito em três etapas: (a) encontrar o nó a ser removido; (b) remover o nó da mesma forma que em uma árvore binária

de busca convencional; e (c) restaurar as propriedades rubro-negras para garantir o balanceamento da estrutura.

Portanto, sua tarefa consiste em implementar um programa que possibilite inserir, buscar e remover elementos em uma árvore rubro-negra. Após cada operação – inserção, busca ou remoção – deverá ser possível visualizar a árvore em formato bidimensional, com indicação da cor de cada nó da árvore.

Além do código-fonte Python você deverá preparar um vídeo, com duração mínima de 10 minutos e máxima de 30 minutos, com uma explicação detalhada da solução implementada. É importante que fique evidente que tal explicação foi apresentada por você, e não por terceiros. Pode ser feito o *upload* do vídeo no AVA (Moodle) ou em uma plataforma de *streaming* de vídeos, como o **YouTube**, e compartilhado o *link* para acesso no arquivo do código-fonte `.py` submetido.

Para a correção do EP (Exercício-Programa) o peso da implementação, em Python, é de 50%, e para o vídeo também é 50%. Vale destacar que, a não entrega do vídeo implicará em nota 0 (zero) tanto para o vídeo quanto para a implementação.

Recomendações importantes

1. A implementação do exercício programa é individual e deve ser obrigatoriamente em linguagem Python 3.14.0 (e testado no ambiente Windows 10, com Visual Studio Code).
2. Além do código-fonte Python, e o vídeo, deve ser entregue um arquivo `.txt` chamado **README** com instruções de como executar o programa, e também exemplos de testes (com entradas e saídas).
3. Não deixe para iniciar a implementação na última hora, como na semana que precede a data limite de entrega.
4. Entregas atrasadas serão permitidas, porém, penalizadas (menos 1 ponto para cada dia de atraso).
5. Cuidado, pois exercícios programas que forem confirmados como **plágio** serão penalizados com nota 0 (zero).

Bom trabalho!